SISTEMAS OPERACIONAIS

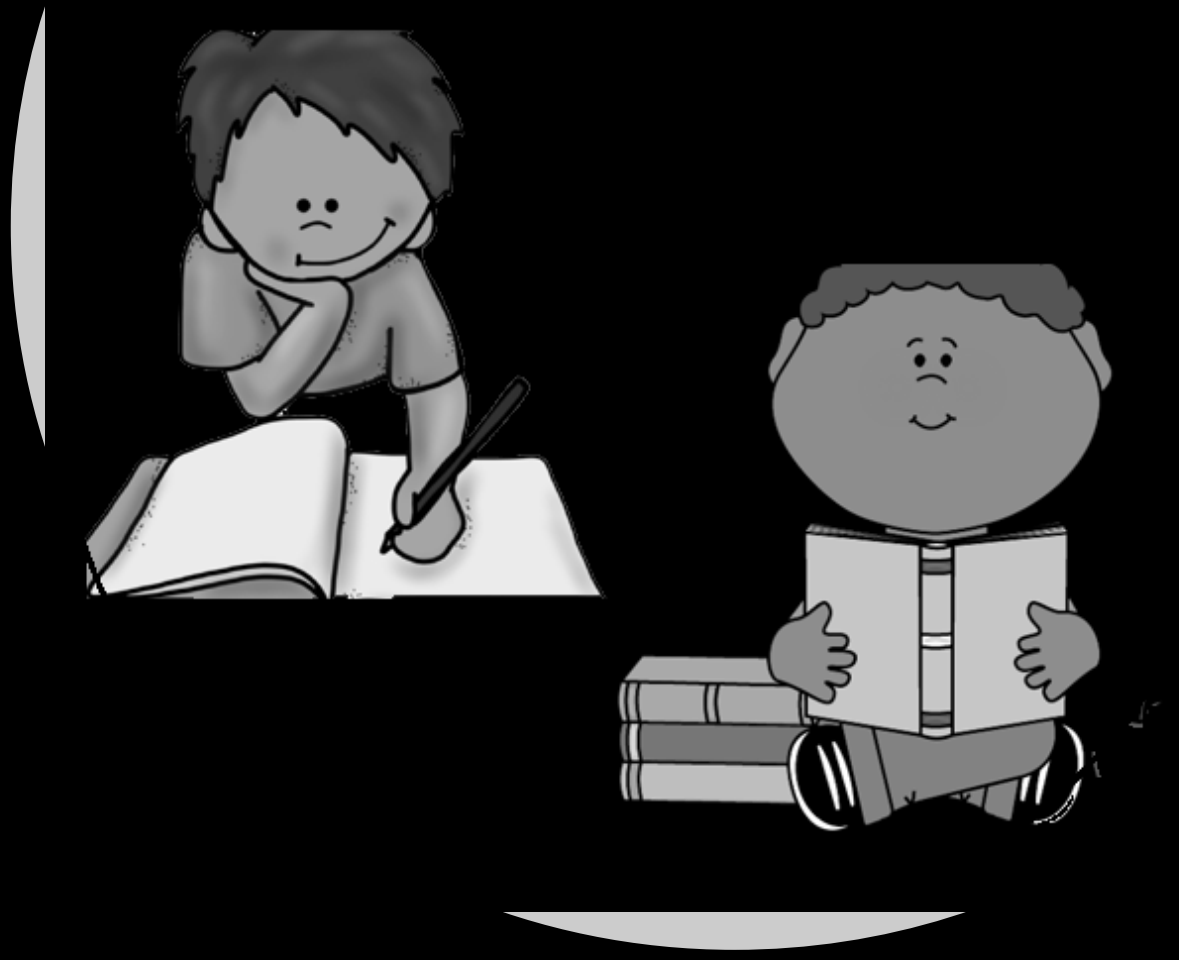
Comunicação Interprocessos: Problemas clássicos

Prof. Matheus A. Souza
Eng. Software/Sist. Informação
PUC Minas

CIP: problemas clássicos

- Cenários que modelam problemas recorrentes em sistemas operacionais e outros sistemas de software
  - Metáforas para componentes e recursos do sistema
- Aplicações em cenários específicos
- Uso das primitivas de sincronização já estudadas

Leitores e Escritores



Leitores e escritores

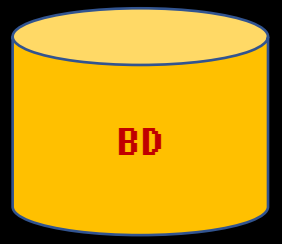
- É comum haver fontes de dados que podem ser consultadas por determinados processos e alteradas por outros
  - Ex: lançamento de notas no Canvas

- Processos leitores
  - É aceitável múltiplas leituras simultâneas

- Processos escritores
  - A escrita é uma operação exclusiva

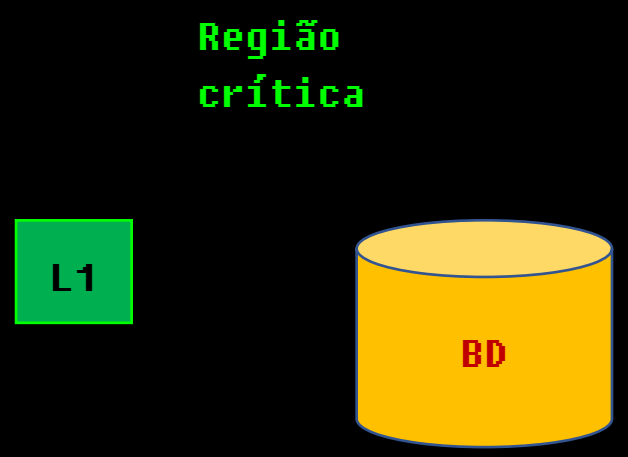
Leitores e escritores

- Premissa para solução básica:
  - Leitores novos podem entrar se já há um ou mais leitores
  - Escritores só entram se os dados estiverem totalmente liberados



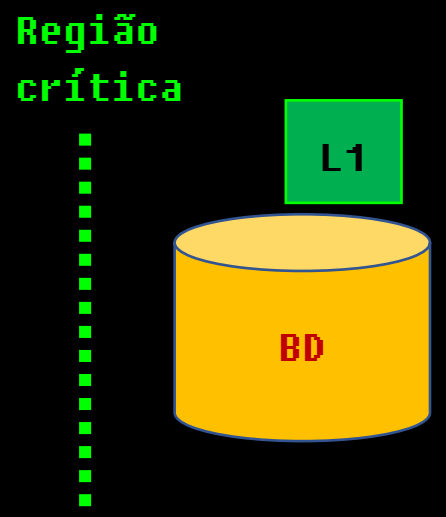
Leitores e escritores

- Premissa para solução básica:
  - Leitores novos podem entrar se já há um ou mais leitores
  - Escritores só entram se os dados estiverem totalmente liberados



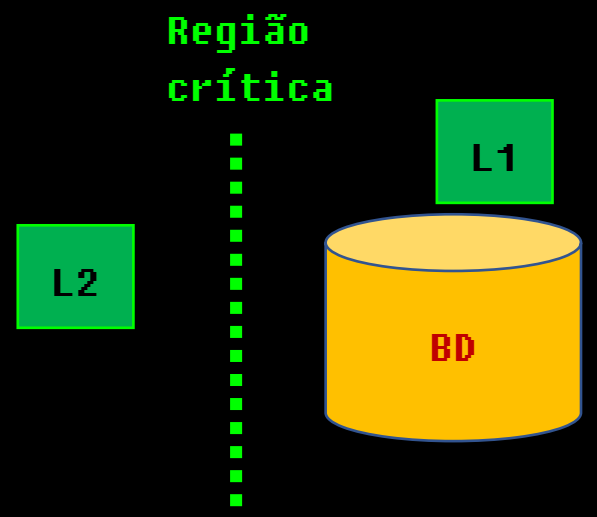
Leitores e escritores

- Premissa para solução básica:
  - Leitores novos podem entrar se já há um ou mais leitores
  - Escritores só entram se os dados estiverem totalmente liberados



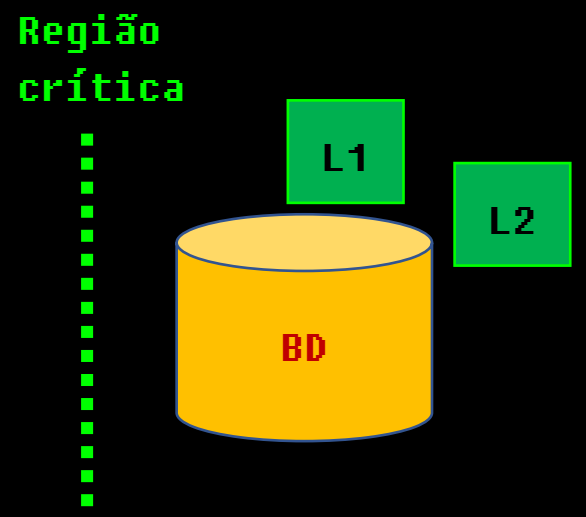
Leitores e escritores

- Premissa para solução básica:
  - Leitores novos podem entrar se já há um ou mais leitores
  - Escritores só entram se os dados estiverem totalmente liberados



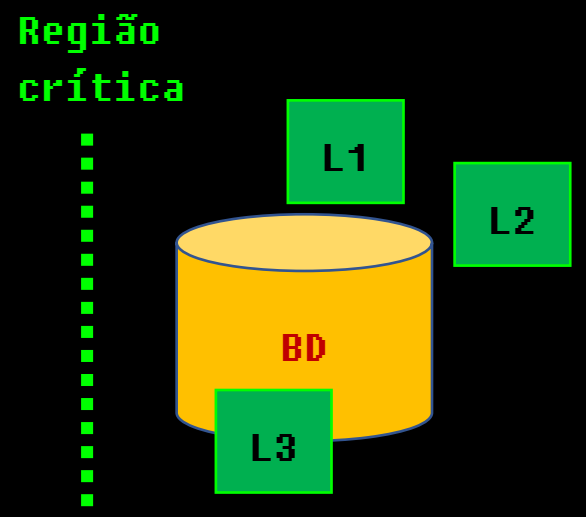
Leitores e escritores

- Premissa para solução básica:
  - Leitores novos podem entrar se já há um ou mais leitores
  - Escritores só entram se os dados estiverem totalmente liberados



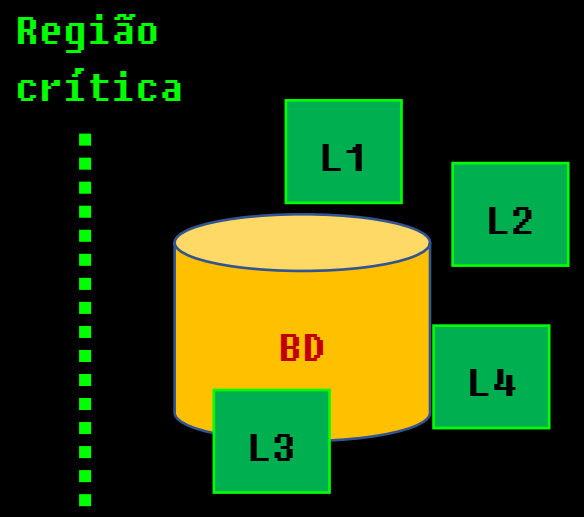
Leitores e escritores

- Premissa para solução básica:
  - Leitores novos podem entrar se já há um ou mais leitores
  - Escritores só entram se os dados estiverem totalmente liberados



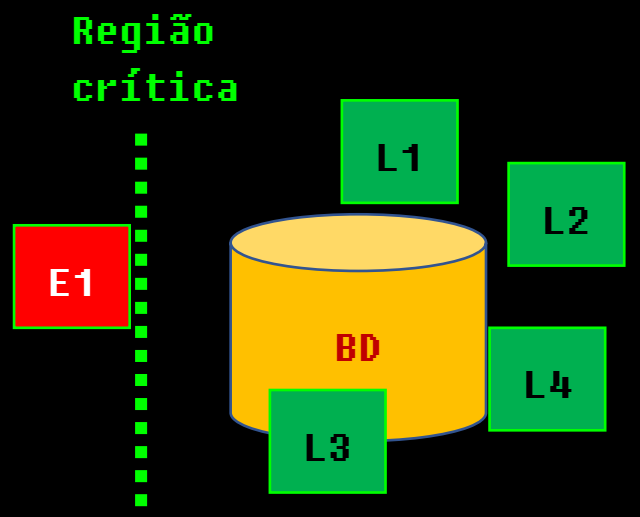
Leitores e escritores

- Premissa para solução básica:
  - Leitores novos podem entrar se já há um ou mais leitores
  - Escritores só entram se os dados estiverem totalmente liberados



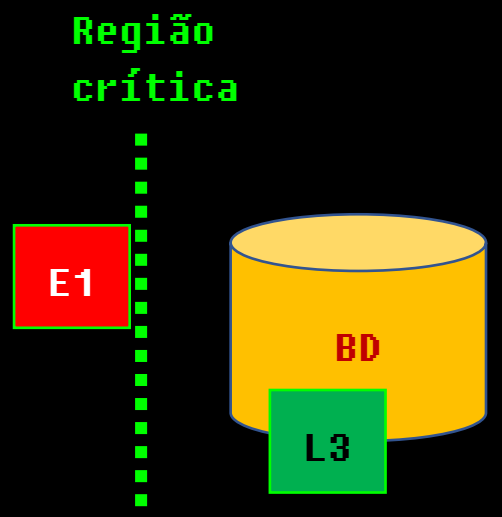
Leitores e escritores

- Premissa para solução básica:
  - Leitores novos podem entrar se já há um ou mais leitores
  - Escritores só entram se os dados estiverem totalmente liberados



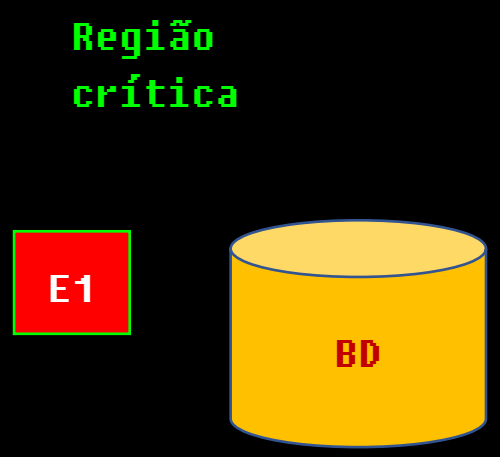
Leitores e escritores

- Premissa para solução básica:
  - Leitores novos podem entrar se já há um ou mais leitores
  - Escritores só entram se os dados estiverem totalmente liberados



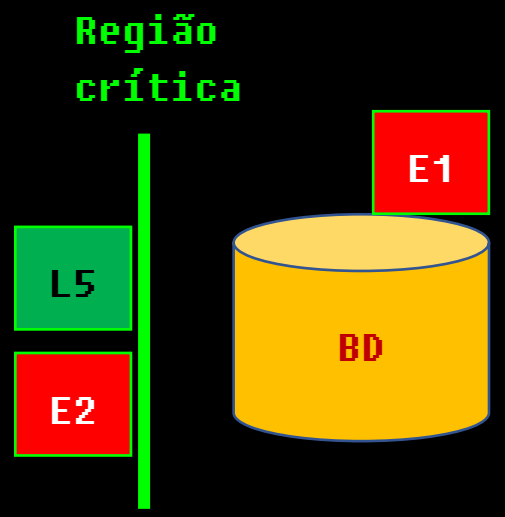
Leitores e escritores

- Premissa para solução básica:
  - Leitores novos podem entrar se já há um ou mais leitores
  - Escritores só entram se os dados estiverem totalmente liberados



Leitores e escritores

- Premissa para solução básica:
  - Leitores novos podem entrar se já há um ou mais leitores
  - Escritores só entram se os dados estiverem totalmente liberados



Leitores e escritores

- Solução com semáforos:
  - Um semáforo para controlar o acesso ao banco de dados
  - Um *mutex* para a região crítica (contador de leitores)

Leitores, escritores e semáforos

```
Semaforo mutex = new Semaforo(1);
Semaforo bd = new Semaforo(1);
int qtLeitores = 0
```

```
leitor(){
    down(mutex);
        qtLeitores++;
        if(qtLeitores==1) down(bd);
    up(mutex);
    ler_dados();
    down(mutex);
        qtLeitores—;
        if(qtLeitores==0) up(bd);
    up(mutex);
}
```

Leitores, escritores e semáforos

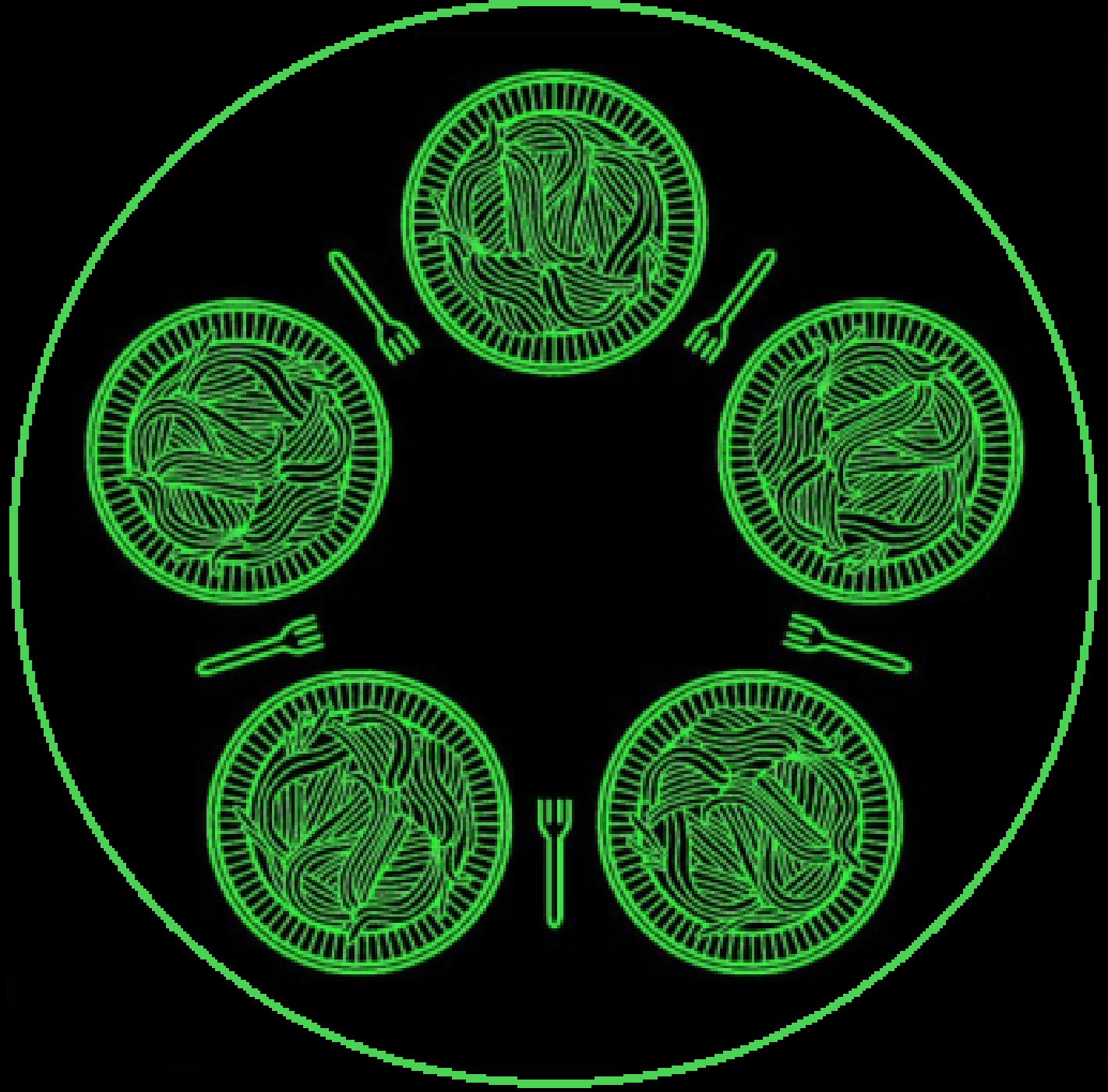
```
Semaforo mutex = new Semaforo(1);
Semaforo bd = new Semaforo(1);
int qtLeitores = 0

escritor(){
    produz_dados();
    down(bd);
        escreve_dados();
    up(bd);
}
```

```
leitor(){
    down(mutex);
        qtLeitores++;
        if(qtLeitores==1) down(bd);
    up(mutex);
    ler_dados();
    down(mutex);
        qtLeitores—;
        if(qtLeitores==0) up(bd);
    up(mutex);
}
```

Leitores, escritores e semáforos

- Esta solução funciona, porém....
  - *Starvation* de escritores
- Proposta de solução:
  - Dar prioridade aos escritores
  - Semáforo adicional de fila de espera
  - Diminui concorrência e garante funcionamento de escritores
  - Escritores costumam ser menos frequentes que leitores



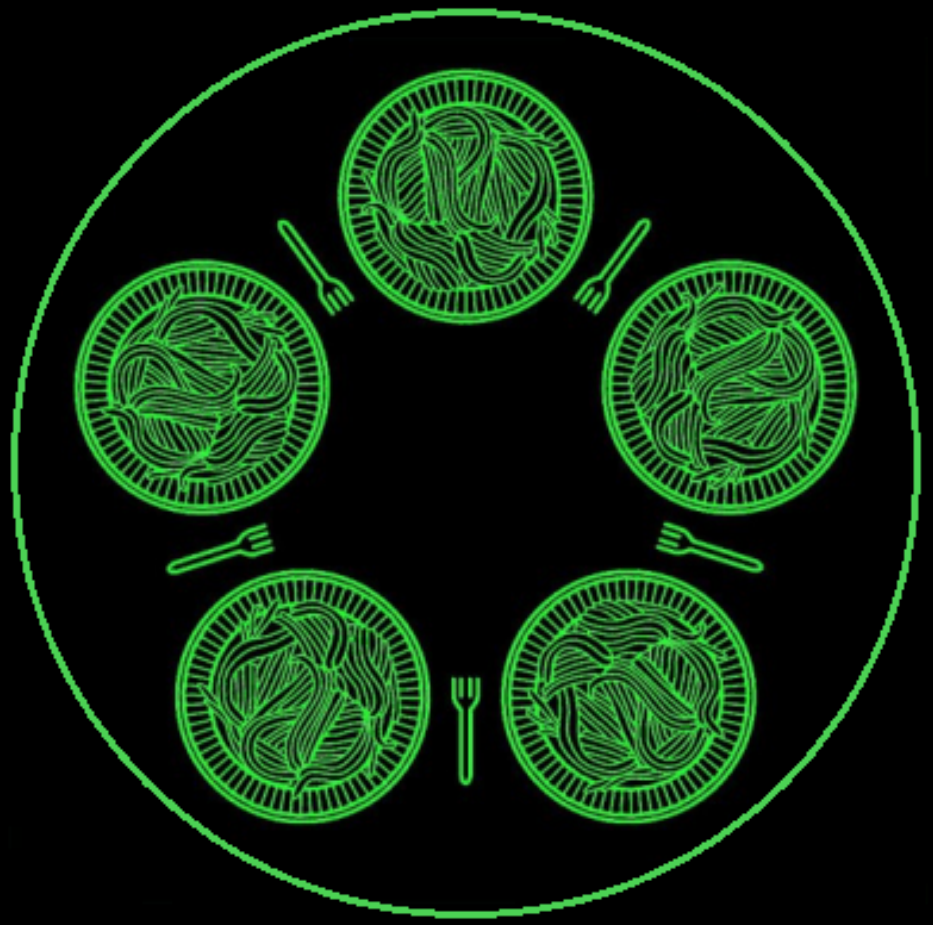
Jantar dos filósofos

Jantar dos filósofos

- Proposto por Dijkstra para demonstrar a efetividade do uso de semáforos

- Cenário:
  - Cinco filósofos em uma mesa circular
  - Filósofos tem duas ações: pensam ou comem
  - Um prato de espaguete (ou arroz?) para cada
  - Cinco talheres, um entre cada par de filósofos
  - Necessidade de dois talheres para comer

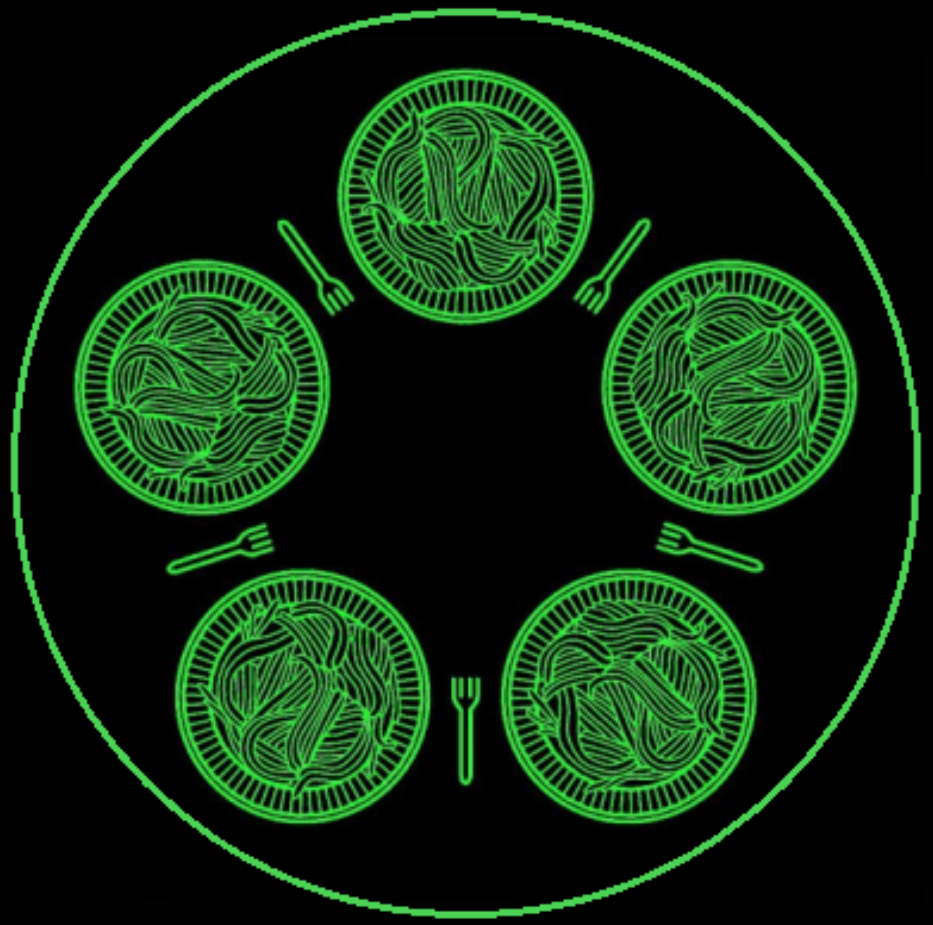
Jantar dos filósofos



- Solução mais óbvia:
  - tentar pegar um garfo de cada vez sendo bloqueado até conseguir
  - caso não consiga, devolva os garfos

- ERRO:
  - tentativas simultâneas podem levar a travamento
  - *Starvation*

Jantar dos filósofos



- Solução aparente: dormir em intervalos aleatórios

- PROBLEMA: não há garantias. E vamos depender de intervalos aleatórios?

Jantar dos filósofos e semáforos

```
#define N            5              /* number of philosophers */
#define LEFT         (i+N–1)%N      /* number of i's left neighbor */
#define RIGHT        (i+1)%N        /* number of i's right neighbor */
#define THINKING     0              /* philosopher is thinking */
#define HUNGRY       1              /* philosopher is trying to get forks */
#define EATING       2              /* philosopher is eating */
typedef int semaphore;              /* semaphores are a special kind of int */
int state[N];                       /* array to keep track of everyone's state */
semaphore mutex = 1;                /* mutual exclusion for critical regions */
semaphore s[N];                     /* one semaphore per philosopher */

void philosopher(int i)             /* i: philosopher number, from 0 to N–1 */
{
    while (TRUE) {                  /* repeat forever */
        think( );                   /* philosopher is thinking */
        take_forks(i);              /* acquire two forks or block */
        eat( );                     /* yum-yum, spaghetti */
        put_forks(i);               /* put both forks back on table */
    }
}
```

Jantar dos filósofos e semáforos

```
void take_forks(int i)          /* i: philosopher number, from 0 to N–1 */
{
    down(&mutex);               /* enter critical region */
    state[i] = HUNGRY;          /* record fact that philosopher i is hungry */
    test(i);                    /* try to acquire 2 forks */
    up(&mutex);                 /* exit critical region */
    down(&s[i]);                /* block if forks were not acquired */
}
```

```
void put_forks(i)               /* i: philosopher number, from 0 to N–1 */
{
    down(&mutex);               /* enter critical region */
    state[i] = THINKING;        /* philosopher has finished eating */
    test(LEFT);                 /* see if left neighbor can now eat */
    test(RIGHT);                /* see if right neighbor can now eat */
    up(&mutex);                 /* exit critical region */
}
```

Jantar dos filósofos e semáforos

```
void test(i) /* i: philosopher number, from 0 to N–1 */
{
        if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
                state[i] = EATING;
                up(&s[i]);
        }
}
```

Jantar dos filósofos e semáforos

- Modelo para situações de competição por acesso exclusivo a recursos compartilhados
  - Ex: uso de várias fontes de dados para gerar um relatório